

ORGNZ-10 LAB: Advanced Segment Definitions - Hands-on Exercises

Series: ORGNZ | **Notebook:** 10 of 10 | **Type:** LAB | **Created:** February 2026 | **Last Updated:** 02/19/2026

Overview

This lab notebook contains 11 hands-on exercises extracted from **ORGNZ-10: Advanced Segment Definitions**. Complete the lecture notebook first, then work through these exercises to reinforce the concepts with real DQL queries against your Dynatrace environment.

Table of Contents

1. [Exercise 1: Supported Operators](#)
 2. [Exercise 2: Supported Operators](#)
 3. [Exercise 3: Primary Grail Fields](#)
 4. [Exercise 4: Primary Grail Fields](#)
 5. [Exercise 5: Primary vs Secondary Variables](#)
 6. [Exercise 6: Primary vs Secondary Variables](#)
 7. [Exercise 7: Primary vs Secondary Variables](#)
 8. [Exercise 8: Primary vs Secondary Variables](#)
 9. [Exercise 9: Using Variables in Filter Conditions](#)
 10. [Exercise 10: Step 2: Create Variable DQL for Each Dimension](#)
 11. [Exercise 11: Step 2: Create Variable DQL for Each Dimension](#)
 12. [Lab Summary](#)
-

Prerequisites

Requirement	Details
Completed	ORGNZ-10: Advanced Segment Definitions (lecture notebook)
Dynatrace Environment	SaaS tenant with Grail enabled
Permissions	logs.read , metrics.read , entities.read , spans.read

Exercise 1: Supported Operators

ORGNZ-10: Advanced Segment Definitions

Series: ORGNZ | **Notebook:** 10 of 10 | **Created:** February 2026 | **Last Updated:** 02/12/2026

This notebook is a deep-dive into the mechanics of creating effective segment definitions in Dynatrace Grail. While **ORGNZ-08** introduced segments, their structure, and design patterns, this notebook focuses on the practical details: filter condition syntax, data type include rules, metadata enrichment for segments, advanced variable patterns, and troubleshooting techniques.

The content draws from real-world implementation patterns, including host group-based segmentation strategies used in enterprise environments.

- 1. Filter Condition Syntax
- 2. Data Type Include Rules
- 3. Primary Grail Fields and Enrichment
- 4. Advanced Variable Patterns
- 5. Host Group-Based Segments

6. Segment Visibility and Sharing
7. Cross-App Integration
8. Known Limitations and Workarounds
9. Troubleshooting and Performance

Requirement	Details
Dynatrace Environment	SaaS environment with Grail enabled
Permissions	<code>storage:filter-segments:read</code> and <code>storage:filter-segments:write</code>
Knowledge	Completed ORGNZ-08 (Grail Segments fundamentals)
Data	At least 1 hour of log, span, and entity data

By the end of this notebook, you will:

- Master filter condition syntax including operators, field access, and combining conditions
- Understand data type include rules and the one-include-per-type constraint
- Know which metadata fields propagate across all signals (and which don't)
- Create advanced variable definitions using DQL entity queries
- Build host group-based segments from real-world naming conventions
- Troubleshoot common segment issues and apply performance best practices

Segment filter conditions define which data a segment includes. Understanding the available operators and field access patterns is critical for building effective segments.

Operator	Example	Notes
<code>=</code> (equals)	<code>k8s.namespace.name = "production"</code>	Exact match, best performance
<code>!=</code> (not equals)	<code>k8s.namespace.name != "kube-system"</code>	Exclude specific values
<code>starts-with</code>	<code>dt.host_group.id = \$platform*</code>	Prefix match with wildcard <code>*</code>
<code>IN</code>	<code>k8s.namespace.name IN ("prod", "staging")</code>	Match any value in list

Operator	Example	Notes
NOT IN	<code>k8s.namespace.name NOT IN ("kube-system")</code>	Exclude list of values

Important — operator availability depends on data type:

- **Signal data includes** (logs, spans, events, metrics) support `=`, `!=`, `contains`, `not contains`, `starts with`, `ends with`, `IN`, and `NOT IN`.
- **Classic entity includes** (`dt.entity.*`) only support `=` (with optional prefix wildcard `*`) and `in()`. `contains` and `ends-with` are **not available** for entity includes.

For cross-signal consistency, prefer prefix-based naming conventions (e.g., `prod-web-tier` not `web-tier-prod`) so `starts-with` works everywhere.

Pattern	Example	Use Case
Direct field	<code>k8s.namespace.name = "production"</code>	Primary Grail Fields
Tag matching	<code>matchesValue(tags, "env:production")</code>	Entity tags
Entity name	<code>entity.name = "my-service"</code>	Specific entity
Host group	<code>dt.host_group.id = "prod-web"</code>	Host group filtering (signal data)
JSON field	<code>content\$.uri = "/health"</code>	Nested JSON in log content

- **AND** — Multiple conditions within a single include are AND-combined:
`k8s.namespace.name = "production" AND k8s.cluster.name = "main-cluster"`
- **OR** — Use the `or` keyword within a single include (you cannot have multiple includes for the same data type):
`k8s.namespace.name = "production" OR k8s.namespace.name = "staging"`

Before creating a segment, always validate your filter conditions with DQL. Run the equivalent filter clause as a DQL query to verify the results match your expectations.

```
// Test segment filter: K8s namespace filtering on logs
// Simulates what a segment with k8s.namespace.name == "production" would
return
fetch logs, from:-1h
| filter isNotNull(k8s.namespace.name)
| summarize count = count(), by:{k8s.namespace.name}
| sort count desc
| limit 20
```

Exercise 2: Supported Operators

```
// Test segment filter: host group filtering on host entities
// Lists hosts and their host group associations (dt.host_group.id is a
signal field, not an entity field)
// Note: Smartscape on Grail uses smartscapeNodes HOST as the preferred
entity query method
fetch dt.entity.host
| expand hg_id = belongs_to[dt.entity.host_group]
| fieldsAdd hg_name = entityName(hg_id, type:"dt.entity.host_group")
| filter isNotNull(hg_id)
| fields entity.name, hg_id, hg_name, tags
| limit 20
```

Exercise 3: Primary Grail Fields

The foundation of effective segment definitions is **metadata enrichment**. In Dynatrace's 3rd-generation architecture, each data point is treated independently. Unlike 2nd-gen where signals were tightly coupled to entities via Management Zones, in 3rd-gen every signal (logs, spans, metrics, events) must be enriched with the corresponding metadata.

Primary Grail Fields are infrastructure-related fields that are **automatically propagated across ALL data types** (spans, logs, metrics, topology). These are the ideal fields for segment filter conditions because they guarantee cross-signal consistency.

Primary Grail Field	DQL Field Name	Source
Host Group	dt.host_group.id	OneAgent host group configuration

Primary Grail Field	DQL Field Name	Source
Kubernetes Cluster	<code>k8s.cluster.name</code>	K8s integration
Kubernetes Namespace	<code>k8s.namespace.name</code>	K8s integration
AWS Account	<code>aws.account.id</code>	AWS integration
Azure Subscription	<code>azure.subscription</code>	Azure integration
GCP Project	<code>gcp.project.id</code>	GCP integration

Best Practice: Always prefer Primary Grail Fields in segment definitions. They are indexed, automatically enriched, and consistent across all signal types.

Some fields go further and propagate to derived data like service metrics:

Field	Propagates to Service Metrics
<code>dt.security_context</code>	Yes
<code>dt.cost.costcenter</code>	Yes
<code>dt.cost.product</code>	Yes
Host group (<code>dt.host_group.id</code>)	Yes
Other host tags	No

When Primary Grail Fields don't align with how your organization wants to segment data (e.g., shared infrastructure with multiple apps on one host group), use **Primary Grail Tags**.

- Identified by the prefix `primary_tags.`
- Example: `primary_tags.stage=prod` , `primary_tags.team=platform`
- Set via host properties, environment variables (`DT_TAGS`), or OpenPipeline
- Propagated across all data points, similar to Primary Grail Fields

Limitation: Primary Grail Tags do not yet enrich Kubernetes metrics or events. They work for logs, spans, and topology data. Check the [Dynatrace documentation](#) for the latest supported signal types.

Scenario	Approach	Effort	Granularity
Dedicated Infrastructure	Host group + host properties	Low	Host-level
Shared Infrastructure	DT_TAGS environment variable per process	Higher	Process-level
Kubernetes	Primary Grail Fields (automatic)	None	Namespace/cluster
Cloud (AWS/Azure/GCP)	Native cloud tags	None	Account/subscription

For dedicated infrastructure, enrich hosts via **Deployment Status** → select hosts → **Modify host properties**:

1. `dt.security_context=<team-or-app>` — For IAM access control
2. `dt.cost.costcenter=<cost-center>` — For cost allocation
3. `dt.cost.product=<product>` — For product tracking
4. `primary_tags.<key>=<value>` — For custom enrichment

Restart Requirements:

- **Logs**: No application restart needed — OneAgent log module auto-enriches after its own restart
- **Spans**: Application restart **required** to apply enrichment to traces
- **Metrics**: OneAgent restart applies automatically

Before building segments, verify that the fields you plan to filter on actually exist on your signal data. Fields with 0% coverage need enrichment via OneAgent configuration or OpenPipeline.

```
// Audit: Check which Primary Grail Fields are populated on log data
// Fields with 0% coverage need enrichment before they can be used in
segments
fetch logs, from:-1h
| summarize
    total = count(),
    with_namespace = countIf(isNotNull(k8s.namespace.name)),
    with_cluster = countIf(isNotNull(k8s.cluster.name)),
    with_host_group = countIf(isNotNull(dt.host_group.id)),
    with_security_ctx = countIf(isNotNull(dt.security_context))
| fieldsAdd
    ns_pct = round(toDouble(with_namespace) / toDouble(total) * 100,
decimals: 1),
    cluster_pct = round(toDouble(with_cluster) / toDouble(total) * 100,
decimals: 1),
    hg_pct = round(toDouble(with_host_group) / toDouble(total) * 100,
decimals: 1),
    sec_pct = round(toDouble(with_security_ctx) / toDouble(total) * 100,
decimals: 1)
```

Exercise 4: Primary Grail Fields

```
// Audit: Verify enrichment fields on span data
// Spans require application restart to pick up new enrichment
fetch spans, from:-1h
| summarize
    total = count(),
    with_namespace = countIf(isNotNull(k8s.namespace.name)),
    with_cluster = countIf(isNotNull(k8s.cluster.name)),
    with_host_group = countIf(isNotNull(dt.host_group.id)),
    with_service = countIf(isNotNull(dt.entity.service))
| fieldsAdd
    ns_pct = round(toDouble(with_namespace) / toDouble(total) * 100,
decimals: 1),
    cluster_pct = round(toDouble(with_cluster) / toDouble(total) * 100,
decimals: 1),
    hg_pct = round(toDouble(with_host_group) / toDouble(total) * 100,
decimals: 1),
    svc_pct = round(toDouble(with_service) / toDouble(total) * 100, decimals:
1)
```


4. Advanced Variable Patterns

Variables make segments dynamic by letting users select values at runtime (e.g., choosing a host group or namespace from a dropdown).

Primary vs Secondary Variables

The variable definition is a DQL query. The columns in the result set determine the variable behavior:

Column Position	Role	Behavior
First column	Primary variable	Shown in the dropdown selector
Additional columns	Secondary variables	Available for use in filter conditions

Variable Rules

Rule	Detail
Maximum values in dropdown	10,000 per variable
Selected values per segment	100 maximum
Wildcard	Allowed after variable name: <code>\$cluster*</code> (starts-with)
No wildcard in names	Variable names and values cannot contain <code>*</code>
Permissions	Users must have read access to entities queried by the variable DQL
Empty dropdown	Usually means the user lacks permission for the variable query

Variable DQL Examples

The following queries demonstrate how to create variable definitions for common use cases.

Exercise 5: Primary vs Secondary Variables

Variables make segments dynamic by letting users select values at runtime (e.g., choosing a host group or namespace from a dropdown).

The variable definition is a DQL query. The columns in the result set determine the variable behavior:

Column Position	Role	Behavior
First column	Primary variable	Shown in the dropdown selector
Additional columns	Secondary variables	Available for use in filter conditions

Rule	Detail
Maximum values in dropdown	10,000 per variable
Selected values per segment	100 maximum
Wildcard	Allowed after variable name: <code>\$cluster*</code> (starts-with)
No wildcard in names	Variable names and values cannot contain <code>*</code>
Permissions	Users must have read access to entities queried by the variable DQL
Empty dropdown	Usually means the user lacks permission for the variable query

The following queries demonstrate how to create variable definitions for common use cases.

```
// Variable DQL: List available host groups
// First column (host_group) = primary variable shown in dropdown
// Second column (id) = secondary variable for filter conditions
// Note: Smartscape on Grail alternative: smartscapeNodes HOST_GROUP
fetch dt.entity.host_group
| fields host_group = entity.name, id
| sort host_group asc
```

Exercise 6: Primary vs Secondary Variables

```
// Variable DQL: List Kubernetes namespaces
// Primary: namespace (dropdown), Secondary: tag (usable in filters)
// Note: Smartscape on Grail alternative: smartscapeNodes K8S_NAMESPACE
fetch dt.entity.cloud_application_namespace
| fields namespace = entity.name
| fieldsAdd tag = concat("Namespace:", namespace)
| sort namespace asc
```

Exercise 7: Primary vs Secondary Variables

```
// Variable DQL: Extract tag values for segment variable creation
// Extracts unique "App" values from host tags with [Environment]App prefix
// Note: Smartscape on Grail alternative: smartscapeNodes HOST
fetch dt.entity.host
| expand tag = tags
| filter startsWith(tag, "[Environment]App")
| parse tag, "LD ':' LD:App"
| fields App, tag
| dedup App
| sort App asc
```

Exercise 8: Primary vs Secondary Variables

```
// Variable DQL: Extract CloudFoundry Organization from process group tags
// Creates variables: $value (org name), $id (process group ID), $AppTag
// (full tag string)
// Note: Smartscape on Grail alternative: smartscapeNodes PROCESS
fetch dt.entity.process_group
| fields tags, id
| expand tags
| parse tags, """"(LD:tag (!&lt;&lt;'\\' ':') LD:value)|LD:tag)""""
| fields tag = replaceString(tag, """"\:"""", ":"), value =
replaceString(value, """"\:"""", ":"), id
| filter toString(tag) == "[CloudFoundry]Organization"
| fields value, id
| fieldsAdd AppTag = concat("[CloudFoundry]Organization:", value)
| dedup value
```

Exercise 9: Using Variables in Filter Conditions

After defining a variable query, reference the columns in your segment filter:

```
Filter: dt.host_group.id = $host_group OR dt.entity.host_group = $id
```

Where `$host_group` references the primary variable (first column) and `$id` references the secondary variable (second column).

Wildcard with variables — Use `$variable*` to match prefixes:

```
Filter: dt.host_group.id = $platform*
```

This matches all host groups that **start with** the selected platform value.

A common enterprise pattern is encoding multiple dimensions into host group names using a naming convention. This section shows how to create segments from these encoded dimensions.

Consider a customer using the host group naming convention

```
<platform>_<app>_<stage> :
```

Host Group	Platform	App	Stage
k8s_multi_prod	k8s	multi	prod
k8s_easytravel_staging	k8s	easytravel	staging
onPrem_easytravel_staging	onPrem	easytravel	staging
onPrem_easytrade_prod	onPrem	easytrade	prod

Create a **separate segment for each dimension** (platform, app, stage). This gives users the flexibility to combine them as needed.

Use DQL to parse the host group name and extract each dimension as a variable:

```
// Parse host group naming convention to extract dimensions
// Convention: <platform>_<app>_<stage>;
// Note: Smartscape on Grail alternative: smartscapeNodes HOST_GROUP
fetch dt.entity.host_group
| parse entity.name, ""LD:platform '_' LD:app '_' LD:stage""
| fields entity.name, platform, app, stage
```

Step 2: Create Variable DQL for Each Dimension

Platform variable:

```
fetch dt.entity.host_group
| parse entity.name, ""LD:platform '_' LD:app '_' LD:stage""
| dedup platform
| fields platform
```

App variable:

```
fetch dt.entity.host_group
| parse entity.name, ""LD:platform '_' LD:app '_' LD:stage""
| dedup app
| fields app
```

Stage variable:

```
fetch dt.entity.host_group
| parse entity.name, ""LD:platform '_' LD:app '_' LD:stage""
| dedup stage
| fields stage
```

Step 3: Define Filter Conditions

Segment	Filter Condition	Notes
Platform	<code>dt.host_group.id = \$platform*</code>	Matches host groups starting with selected platform
App	<code>dt.host_group.id contains \$app</code>	Matches host groups containing selected app (signal includes only)
Stage	<code>dt.host_group.id ends-with _\$stage</code>	Matches host groups ending with selected stage (signal includes only)

Note: `contains` and `ends-with` operators are available for signal data includes (logs, spans, events, metrics) but **not** for classic entity includes. For entity includes, use `starts-with` (prefix wildcard) patterns. Always test with the segment preview before deploying.

Step 4: Validate Across Signal Types

After creating the segments, validate they filter correctly across:

- **Logs** — Check record count reduction when segment is applied
- **Distributed Traces** — Verify spans are filtered by the host group dimension
- **Hosts / Services** — Confirm entity filtering matches expectations
- **Infrastructure** — Verify metrics are scoped correctly

Tip: The host group is a Primary Grail Field, so it propagates across all signal types. This makes it an excellent candidate for segment filtering.

6. Segment Visibility and Sharing

Visibility Settings

Setting	Behavior
Unlisted (default)	Only visible to the creator and users it's shared with

Setting	Behavior
Public	Visible in everyone's segment list in the environment

Permissions Model

Permission	Action	Included In
<code>storage:filter-segments:read</code>	View and use segments	Dynatrace Standard User
<code>storage:filter-segments:write</code>	Create and edit segments	Dynatrace Standard User
<code>storage:filter-segments:share</code>	Share with others	Dynatrace Standard User
<code>storage:filter-segments:delete</code>	Delete segments	Dynatrace Standard User
<code>storage:filter-segments:admin</code>	Manage segment permissions	Dynatrace Professional User

Governance Recommendations

- **Platform team** owns environment and infrastructure segments (public)
- **Application teams** own app-specific segments (shared with their group)
- Maintain a small set of "official" public segments
- Use consistent naming: `team-`, `env-`, `app-`, `region-` prefixes

7. Cross-App Integration

Cross-App Persistence

When you select a segment in one Dynatrace app (e.g., Logs), the selection **persists** as you navigate to other apps (e.g., Distributed Traces, Problems). This means you don't need to reapply filters when drilling into different data types during an investigation.

App Support Matrix

App	Segment Support	Notes
Dashboards	Full	Dashboard-level and tile-level segments
Notebooks	Full	Segment selector at top of notebook
Logs	Full	Filters all log queries
Distributed Traces	Full	Filters span queries
Problems	Full	Requires events include with <code>event.kind = "DAVIS_PROBLEM"</code>
SLOs	Full	Filters SLO evaluation scope
Site Reliability Guardian	Full	Filters validation scope
Workflows	Full	Available in workflow DQL steps
Smartscape	Full	Filters topology view

Dashboard Segment Behavior

- **Dashboard-level segment** applies to all tiles
- **Tile-level segment** overrides the dashboard segment for that specific tile
- Use this pattern for executive dashboards that show cross-team metrics alongside team-specific views

8. Known Limitations and Workarounds

Limitation	Detail	Workaround
detected problems require event includes	To filter problems with segments, define an events include with <code>event.kind = "DAVIS_PROBLEM"</code> ; entity includes alone do not filter problem records	Add event-type includes alongside entity includes

Limitation	Detail	Workaround
Single relationship traversal	Entity relationships can only traverse one hop	Target specific entity types directly in includes
Inclusions only, no exclusions	Cannot say "everything EXCEPT team-X"	Explicitly include what you want (MZ supported exclusions; segments do not)
Entity includes: no contains/ends-with	Classic entity includes only support <code>=</code> and prefix wildcards; <code>contains</code> and <code>ends-with</code> are not available	Use prefix-based naming conventions; signal includes support broader operators
1 include per data type	Cannot have two <code>logs</code> include blocks	Combine conditions with <code>OR</code> within a single include
Max 20 includes per segment	Hard limit on rule count	Consolidate rules; use variables for flexibility
Max 10 expressions per filter	Limits filter complexity	Use OpenPipeline to pre-enrich data with simpler filter fields
Max 10 segments per query	Cannot stack unlimited segments	Design broader segments instead of many narrow ones
Variable dropdown empty	Users without entity read permissions see empty dropdown	Ensure variable DQL targets entities the user can read

9. Troubleshooting and Performance

Common Issues

Symptom	Likely Cause	Fix
Empty variable dropdown	Missing entity read permissions	Grant <code>dt.entity.*:read</code> to user's policy

Symptom	Likely Cause	Fix
Segment shows no data	Filter field not enriched on signal data	Verify enrichment with audit queries from Section 3
Segment filters entities but not logs	Entity includes don't apply to signals	Add separate signal-type includes (logs, spans)
Unexpected data included	OR logic combining values	Review filter condition logic within the include
Segment missing in app	Segment set to unlisted and not shared	Share segment or set visibility to public
Segment works for logs but not spans	Field exists on logs but not spans	Use Primary Grail Fields for cross-signal consistency

Performance Best Practices

Every segment filter condition is injected into every query the user runs. Keep conditions simple:

1. **Prefer Primary Grail Fields** — They are indexed and optimized for filtering
2. **Use exact matches** — `=` is faster than `starts-with`
3. **Minimize expressions** — Fewer conditions = better query performance
4. **Use OpenPipeline pre-processing** — If conditions would be complex, pre-enrich a simple field (like `dt.security_context`) and filter on that instead
5. **Narrow scope first** — Start with the most restrictive condition in AND combinations

Naming Conventions

Prefix	Use Case	Examples
<code>team-</code>	Team-based segments	<code>team-platform-infra</code> , <code>team-checkout</code>
<code>env-</code>	Environment segments	<code>env-production</code> , <code>env-staging</code>
<code>app-</code>	Application segments	<code>app-checkout-full</code> , <code>app-payments</code>
<code>region-</code>	Regional segments	<code>region-us-east</code> , <code>region-eu-west</code>

Segment Lifecycle

1. **Design** — Identify data types, filter conditions, variables
2. **Test** — Validate filter logic with DQL queries
3. **Create** — Build in the Segments app
4. **Share** — Distribute to appropriate users/groups
5. **Monitor** — Check for empty results, permission issues
6. **Retire** — Remove when no longer needed; review quarterly

Discover Available Filter Candidates

Exercise 10: Step 2: Create Variable DQL for Each Dimension

Platform variable:

```
fetch dt.entity.host_group
| parse entity.name, "LD:platform '_' LD:app '_' LD:stage"
| dedup platform
| fields platform
```

App variable:

```
fetch dt.entity.host_group
| parse entity.name, "LD:platform '_' LD:app '_' LD:stage"
| dedup app
| fields app
```

Stage variable:

```
fetch dt.entity.host_group
| parse entity.name, "LD:platform '_' LD:app '_' LD:stage"
| dedup stage
| fields stage
```

Segment	Filter Condition	Notes
Platform	<code>dt.host_group.id = \$platform*</code>	Matches host groups starting with selected platform
App	<code>dt.host_group.id contains \$app</code>	Matches host groups containing selected app (signal includes only)
Stage	<code>dt.host_group.id ends-with _\$stage</code>	Matches host groups ending with selected stage (signal includes only)

Note: `contains` and `ends-with` operators are available for signal data includes (logs, spans, events, metrics) but **not** for classic entity includes. For entity includes, use `starts-with` (prefix wildcard) patterns. Always test with the segment preview before deploying.

After creating the segments, validate they filter correctly across:

- **Logs** — Check record count reduction when segment is applied
- **Distributed Traces** — Verify spans are filtered by the host group dimension
- **Hosts / Services** — Confirm entity filtering matches expectations
- **Infrastructure** — Verify metrics are scoped correctly

Tip: The host group is a Primary Grail Field, so it propagates across all signal types. This makes it an excellent candidate for segment filtering.

Setting	Behavior
Unlisted (default)	Only visible to the creator and users it's shared with
Public	Visible in everyone's segment list in the environment

Permission	Action	Included In
<code>storage:filter-segments:read</code>	View and use segments	Dynatrace Standard User
<code>storage:filter-segments:write</code>	Create and edit segments	Dynatrace Standard User
<code>storage:filter-segments:share</code>	Share with others	Dynatrace Standard User

Permission	Action	Included In
<code>storage:filter-segments:delete</code>	Delete segments	Dynatrace Standard User
<code>storage:filter-segments:admin</code>	Manage segment permissions	Dynatrace Professional User

- **Platform team** owns environment and infrastructure segments (public)
- **Application teams** own app-specific segments (shared with their group)
- Maintain a small set of "official" public segments
- Use consistent naming: `team-`, `env-`, `app-`, `region-` prefixes

When you select a segment in one Dynatrace app (e.g., Logs), the selection **persists** as you navigate to other apps (e.g., Distributed Traces, Problems). This means you don't need to reapply filters when drilling into different data types during an investigation.

App	Segment Support	Notes
Dashboards	Full	Dashboard-level and tile-level segments
Notebooks	Full	Segment selector at top of notebook
Logs	Full	Filters all log queries
Distributed Traces	Full	Filters span queries
Problems	Full	Requires events include with <code>event.kind = "DAVIS_PROBLEM"</code>
SLOs	Full	Filters SLO evaluation scope
Site Reliability Guardian	Full	Filters validation scope
Workflows	Full	Available in workflow DQL steps
Smartscape	Full	Filters topology view

- **Dashboard-level segment** applies to all tiles
- **Tile-level segment** overrides the dashboard segment for that specific tile
- Use this pattern for executive dashboards that show cross-team metrics alongside team-specific views

Limitation	Detail	Workaround
detected problems require event includes	To filter problems with segments, define an events include with <code>event.kind = "DAVIS_PROBLEM"</code> ; entity includes alone do not filter problem records	Add event-type includes alongside entity includes
Single relationship traversal	Entity relationships can only traverse one hop	Target specific entity types directly in includes
Inclusions only, no exclusions	Cannot say "everything EXCEPT team-X"	Explicitly include what you want (MZ supported exclusions; segments do not)
Entity includes: no contains/ends-with	Classic entity includes only support = and prefix wildcards; contains and ends-with are not available	Use prefix-based naming conventions; signal includes support broader operators
1 include per data type	Cannot have two <code>logs</code> include blocks	Combine conditions with <code>OR</code> within a single include
Max 20 includes per segment	Hard limit on rule count	Consolidate rules; use variables for flexibility
Max 10 expressions per filter	Limits filter complexity	Use OpenPipeline to pre-enrich data with simpler filter fields
Max 10 segments per query	Cannot stack unlimited segments	Design broader segments instead of many narrow ones
Variable dropdown empty	Users without entity read permissions see empty dropdown	Ensure variable DQL targets entities the user can read

Symptom	Likely Cause	Fix
Empty variable dropdown	Missing entity read permissions	Grant <code>dt.entity.*:read</code> to user's policy

Symptom	Likely Cause	Fix
Segment shows no data	Filter field not enriched on signal data	Verify enrichment with audit queries from Section 3
Segment filters entities but not logs	Entity includes don't apply to signals	Add separate signal-type includes (logs, spans)
Unexpected data included	OR logic combining values	Review filter condition logic within the include
Segment missing in app	Segment set to unlisted and not shared	Share segment or set visibility to public
Segment works for logs but not spans	Field exists on logs but not spans	Use Primary Grail Fields for cross-signal consistency

Every segment filter condition is injected into every query the user runs. Keep conditions simple:

1. **Prefer Primary Grail Fields** — They are indexed and optimized for filtering
2. **Use exact matches** — `=` is faster than `starts-with`
3. **Minimize expressions** — Fewer conditions = better query performance
4. **Use OpenPipeline pre-processing** — If conditions would be complex, pre-enrich a simple field (like `dt.security_context`) and filter on that instead
5. **Narrow scope first** — Start with the most restrictive condition in AND combinations

Prefix	Use Case	Examples
<code>team-</code>	Team-based segments	<code>team-platform-infra</code> , <code>team-checkout</code>
<code>env-</code>	Environment segments	<code>env-production</code> , <code>env-staging</code>
<code>app-</code>	Application segments	<code>app-checkout-full</code> , <code>app-payments</code>
<code>region-</code>	Regional segments	<code>region-us-east</code> , <code>region-eu-west</code>

1. **Design** — Identify data types, filter conditions, variables
2. **Test** — Validate filter logic with DQL queries
3. **Create** — Build in the Segments app
4. **Share** — Distribute to appropriate users/groups
5. **Monitor** — Check for empty results, permission issues
6. **Retire** — Remove when no longer needed; review quarterly

```
// Discover: Find the most common tag keys across hosts
// Use these to plan segment filter conditions
// Note: Smartscape on Grail alternative: smartscapeNodes HOST
fetch dt.entity.host
| expand tag = tags
| summarize count = count(), by:{tag}
| sort count desc
| limit 30
```

Exercise 11: Step 2: Create Variable DQL for Each Dimension

```
// Test: Simulate a combined segment filter on logs
// Combines host group and namespace filtering to verify expected results
fetch logs, from:-1h
| filter isNotNull(dt.host_group.id) and isNotNull(k8s.namespace.name)
| summarize count = count(), by:{dt.host_group.id, k8s.namespace.name}
| sort count desc
| limit 20
```

Lab Summary

You have completed 11 hands-on exercises for **ORGNZ-10: Advanced Segment Definitions**.

Exercises Completed

- [] Exercise 1: Supported Operators
- [] Exercise 2: Supported Operators
- [] Exercise 3: Primary Grail Fields
- [] Exercise 4: Primary Grail Fields
- [] Exercise 5: Primary vs Secondary Variables
- [] Exercise 6: Primary vs Secondary Variables
- [] Exercise 7: Primary vs Secondary Variables
- [] Exercise 8: Primary vs Secondary Variables

- [] Exercise 9: Using Variables in Filter Conditions
- [] Exercise 10: Step 2: Create Variable DQL for Each Dimension
- [] Exercise 11: Step 2: Create Variable DQL for Each Dimension

Next Steps

Congratulations! You have completed the full ORGNZ series.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.